42 MULHOUSE

Les bases du C

... ou pas

Le binaire sur 8 bits

Chaque bit vaut une puissance de 2.

Décimal	Binaire
0	0000 0000
1	0000 0001
2	0000 0010
3	0000 0011
4	0000 0100
5	0000 0101

128	64	32	16	8	4	2	1
0	0	0	1	0	1	0	0

0001 0100 = 16 + 4 = 20

Les types

Type	Taille	Max (non signé)	Max (signé)
char	1	255	127
short	2	65 535	32 767
int	4	~4,2 milliards	~2,1 milliards

> En signé, un bit sert au signe : le maximum est divisé par deux.

`sizeof(type)` donne la taille d'un type, utile pour malloc.

Les symboles

Symbole	Rôle
*	déclarer un pointeur, ou lire la valeur pointée
&	récupérer l’adresse d’une variable

```
int a = 42;
int *ptr = &a;         // on prend l'adresse de a
printf("%d", *ptr);    // on affiche la valeur pointée : 42
```

Stack et heap

Stack

- gérée par le compilateur
- taille limitée
- libérée automatiquement

```
char str[5] = "chat";
```

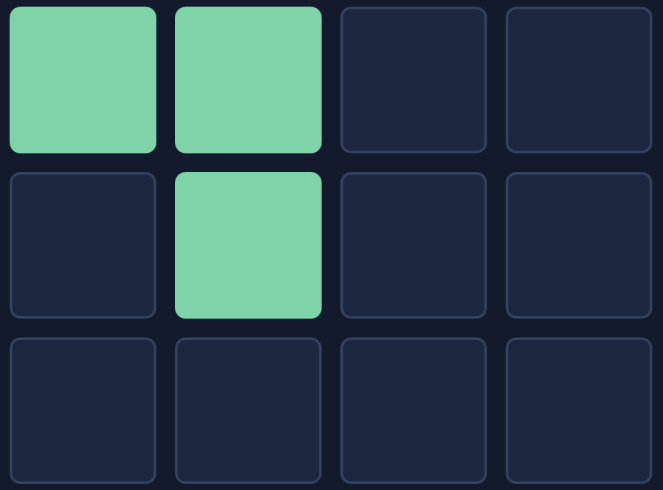
Heap

- gérée par le développeur
- taille flexible (la RAM)
- doit être libérée

```
char *str = malloc(5);
if (!str)
    return (NULL);
free(str);
```

La mémoire

Le système donne la mémoire par pages de 4096 octets.



Une page est donnée entière

Même si tu n'utilises qu'un octet.

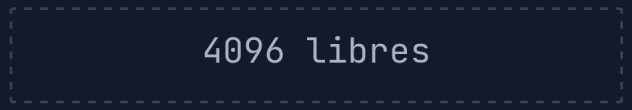
Une variable non initialisée contient d'anciennes valeurs

Laissées par ton programme : toujours initialiser.

Malloc arena

malloc demande une page au système, puis la découpe lui-même.

Le système donne 4096 octets



malloc garde la page pour lui.

malloc(100)



Le reste sert aux malloc suivants.

Page pleine



malloc redemande une page.

int nbr = 5

Un int occupe 4 octets, de 0x1000 à 0x1003.

Adresse	Octet
0x0FFF	???? ????
0x1000	0000 0101
0x1001	0000 0000
0x1002	0000 0000
0x1003	0000 0000
0x1004	???? ????

Relu dans le bon ordre :

`0000 0000 0000 0000 0000 0000 0000 0101`

`= 5`

Les octets sont rangés « à l'envers » : c'est le little-endian.

Réassignation : nbr = 6

La variable ne bouge pas : seule la valeur change.

Adresse	Octet
0x0FFF	???? ????
0x1000	0000 0101
0x1001	0000 0000
0x1002	0000 0000
0x1003	0000 0000
0x1004	???? ????

→

Adresse	Octet
0x0FFF	???? ????
0x1000	0000 0110
0x1001	0000 0000
0x1002	0000 0000
0x1003	0000 0000
0x1004	???? ????

Et les négatifs ?

Exemples sur 8 bits pour que ce soit lisible.

On pourrait croire : `1000 0001 = -1` Ben non...

En vrai : `1111 1111 = -1`

> La règle : on **inverse tous les bits**, puis on **ajoute 1**.

`1 = 0000 0001  →  1111 1110  →  1111 1111 = -1`

Pourquoi cette écriture ?

Parce que l'addition normale marche directement.

```
  1111 1111   (-1)
+ 0000 0001   (1)
= 1 0000 0000
```

Le 1 qui dépasse est ignoré : il reste 0.

```
  0000 1010   (10)
+ 1110 0111   (-25)
= 1111 0001   (-15)
```

Ça marche aussi avec n'importe quel nombre.

En binaire : 0 + 0 = 0, 0 + 1 = 1, 1 + 1 = 0 et je retiens 1.

Et les multiplications ?

Décaler d'un bit vers la gauche, c'est multiplier par 2.

```
3 × 3
= (3 × 2) + 3
= (3 << 1) + 3
```

```
  0000 0110   (3 << 1 = 6)
+ 0000 0011   (3)
= 0000 1001   (9)
```

On divise ?

Décaler d'un bit vers la droite, c'est diviser par 2.

```
  0000 0101   (5)
>> 1
= 0000 0010   (2)
```

5 / 2 = 2, comme en C

La division entière garde le quotient.

Le bit qui sort (1) est le reste

5 % 2 = 1

Le poids des bits

ET : &

```
  0000 1010   (10)
& 0000 0001   (1)
= 0000 0000   (0)
```

1 seulement si les deux bits valent 1. Ici 0 : 10 est pair.

OU : |

```
  0000 1010   (10)
| 0000 0001   (1)
= 0000 1011   (11)
```

1 si au moins un des bits vaut 1.

> Petit test : que donne **10 & −1** ?

Bits swap

Échanger les deux moitiés d'un nombre.

```
n          = 0x1234 5678
n << 16    = 0x5678 0000
n >> 16    = 0x0000 1234
gauche | droite = 0x5678 1234
```

> On décale une moitié à gauche, l'autre à droite, puis | les recolle. Utilise un **unsigned int**.

char c = ’a’

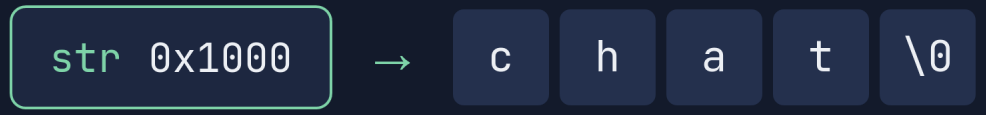
Un char = 1 octet. On y range le code ASCII de la lettre.

Adresse	Octet	ASCII	Caractère
0x0FFF	???? ????		
0x1000	0110 0001	97	a
0x1001	???? ????		

'a' = 97 = 0110 0001

char *str = ”chat”

str contient seulement l’adresse du premier caractère.



5 octets pour la chaîne

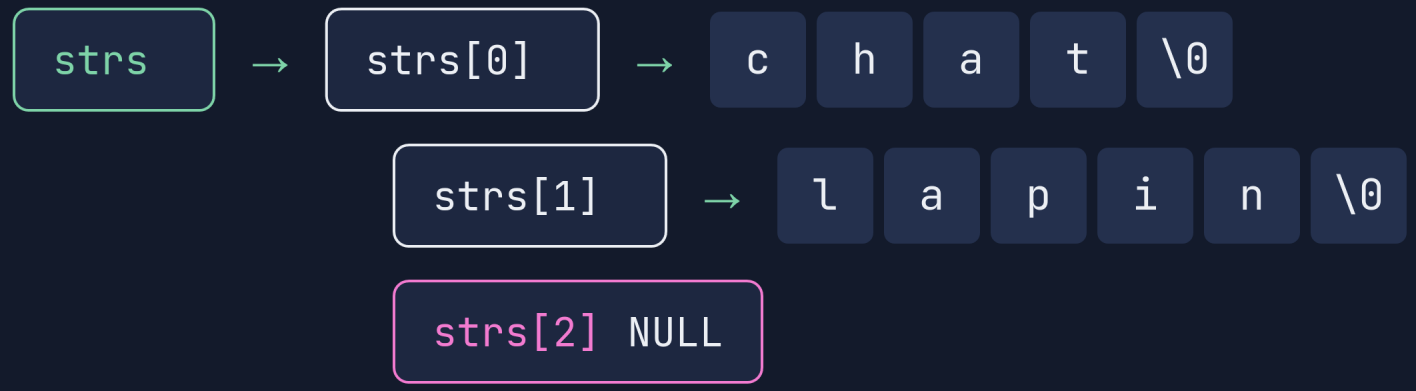
4 lettres + le \0 qui marque la fin.

8 octets pour str

C’est la taille d’une adresse.

char **strs

Un tableau d'adresses, terminé par NULL.



1re * : choisit le mot **2e * : parcourt le mot** **NULL marque la fin**

Alignement mémoire (padding)

Le processeur lit la mémoire par blocs : un int doit commencer sur un multiple de 4.



c = char · i = int · « · » = octets vides

> sizeof(struct s) = **8**, et pas 5 : le compilateur ajoute 3 octets vides.

Adresse et masque ~7

Une adresse alignée sur 8 a ses 3 derniers bits à 0.

```
  7 = 0000 … 0111
~7 = 1111 … 1000
```

> **addr & ~7** force les 3 derniers bits à 0 : on obtient un multiple de 8.

```
0x1003 & ~7 = 0x1000
```

Merci !

Des questions ?

eschwart · 42 Mulhouse